SIMATSENGINEERING

Department of Information Technology

DSA01–Object-OrientedProgrammingwithC++

ASSIGNMENT1–CO1

Design,Implement and Validate the Fare Logic of a City Metro Smart-Card System

CO1:Applyobject-orientedprogrammingprinciplesandC++constructssuchasdatatypes,operators and control structures to design and write structured OOP-based programs.

Student Name	R. Kishan Raghava	**Register Number**	192525154
Department / Section	AI&ML	**Academic Year / Semester**	2025
Faculty Name	**AMIRTHA S**	**Natureof Assignment**	Individual
Duration	1Week	**Total Marks**	50
Date of Submission	06-11-2026	**Date of Evaluation**	
GitHub Repository	192525154-DSA0114	**Compiler**	g++13.3.0,-std=c++17-Wall-Wextra

MARK SHEET(Rubric-basedEvaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Object-Oriented Analysis	4		
2	Tokens, Keywords, Identifiers and Constants	3		
3	Data Types and Compatibility	4		
4	Implicit Conversions	4		
5	Operators and Precedence	4		
6	Control Structures	4		
7	Procedural vs Object-Oriented	4		
8	Inputand Output	4		
9	Implementation and Testing	4		
10	Expectation vs Execution	4		
11	Final Recommendation	4		
12A	Re-evaluate: Impact Analysis	2		
12B	Re-evaluate: Evaluation of Original Design	2		
12C	Re-evaluate: Adaptation or Redesign	2		
12D	Re-evaluate: Evidence-Based Defence	2		
	TOTAL	50		Percentage:

OVERALLREMARKS

R.kishanraghava		
Student Signature	**Faculty Signature**	**Program Director**

OBJECT-ORIENTEDPROGRAMMINGWITHC++

ASSIGNMENT 1 – CO1

CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs – the object-oriented paradigm and itsbasic principles, tokens,keywords, identifiers, constants, data types,type compatibility,variables,operators,implicitconversions,operatorprecedenceandcontrolstructures.

Nature of Assignment: Individual
Duration:1Week
TotalMarks:50
Submission Date: _____
Bloom'sLevelsAssessed:K5–Evaluate(everytaskinthisassignmentisatK5level)

AssignmentTitle:Design,ImplementandValidatetheFareLogicofaCityMetroSmart-Card System (Total Marks - 50)

Mandatory Execution and GitHub Submission

- Every student must individuallyexecuteeverytaskinthisassignment,includingall"Complete the Code" programs.
- Create a Git Hub repository named DSA01-<Register Number>-Assignment1 and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for eacht ask(Task01,Task02…),containing theC++sourcefiles,input files and output screenshots.
- IncludeaREADME.mdwithyourname,registernumber,compilerandversion,theexact g++ commandusedtocompile, thestepstoruneachprogram, andashort noteoftheresultofeach task.
- Commit your work progressively – at least one commit for each task. A single upload on the last day will not be accepted.
- SubmittheGitHublinkbelowandonthecover pageofyour report.

GitHub Repository Link: _____

Howto AnswertheAdditionalQuestions

Complete the Code: Copythe given code into your repository, replace everyTODO with working codewithoutchangingtherestoftheprogram,executeitandcompareyouroutputwiththeexpected output.

Predict the Output: Writethe exact output ofthe givenprogrambefore running it. Thenrun it and explaineverydifferencebetweenyourpredictionandtheactualoutput,usingtheC++rulesinvolved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario—NammaMetroSmart-CardFare System

A city metro rail divides its network into three zones – Inner, Middle and Outer. The fare is Rs 10, Rs20 or Rs30 according tothe farthest zonetravelled, witha20% surchargeduring peak hours(8–10 AM and 5–8 PM). Passengers use a stored-value smart card, and a journey is allowed only if at least Rs 50 remains on the card after the fare is deducted. Students and senior citizens receive a concession.

The first version ofthe gatesoftware was written in a hurry. Some fares are off bya paisa, the peak checkaccepts10AMaspeakand8PMasoff-peakinconsistently,anegativebalanceonceappeared on a card, and the fare values are typed in several places.

Thenumberofjourneysprocessed maygrow:1station(20,000journeys aday)→1 line(500,000a day) → whole network (3,000,000 a day).

You are required to analyse the problem, identify its objects, choose suitable C++ data types, constants,operatorsandcontrolstructures,implementthefarelogicandvalidateit,sothateveryfare and balance is exact.

Design Challenge:

The metro is adding a fourth zone, a weekend discount and monthly passes. Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

Introduction

The City Metro Smart-Card System is designed to automate fare calculation and balance management for metro passengers. The system calculates journey fares based on travel zones, peak-hour timings and passenger concessions. It also ensures that passengers maintain a minimum balance of ₹50 after every journey.

The metro network initially contains three zones: Inner Zone, Middle Zone and Outer Zone. Each zone has a different base fare. During peak hours, an additional 20% surcharge is applied to the base fare. Peak hours are from 8:00 AM to 10:00 AM and from 5:00 PM to 8:00 PM.

The system must be designed using Object-Oriented Programming concepts in C++. It should provide accurate fare calculations, safe balance deductions, proper input validation and reliable handling of a large number of journeys.

Problem Statement

The existing metro fare system has several problems, including incorrect fare calculations, boundary-time errors, negative card balances and duplicated fare values in different parts of the program. These problems can lead to incorrect passenger charges and financial losses. The objective is to design and implement a reliable C++ fare management system using suitable data types, constants, control structures and classes. The system must calculate fares accurately, validate passenger inputs, maintain the minimum balance requirement and support future expansion of the metro network.

Objectives

1. To understand and apply Object-Oriented Programming concepts in C++.
2. To implement accurate fare calculation using constants and suitable data types.
3. To identify and prevent errors caused by type conversions and operator precedence.
4. To use conditional statements and loops for journey processing.
5. To protect smart-card balance using encapsulation.
6. To validate user input and generate formatted journey reports.
7. To test normal, boundary and invalid journey conditions.
8. To evaluate the system's performance for large journey volumes.
9. To recommend a scalable design for the metro network.
10. To extend the system to support additional zones, weekend discounts and monthly passes.

System Requirements

Zone	Base fare
Inner Zone	₹10
Middle Zone	₹20
Outer Zone	₹30

- Peak-hour surcharge: 20%.
- Morning peak period: 8:00 AM to before 10:00 AM.
- Evening peak period: 5:00 PM to before 8:00 PM.
- Minimum balance required after fare deduction: ₹50.
- Passenger concessions: student and senior concessions are considered in the model solution; the exact percentage must be confirmed because it is not specified in the original assignment.

General Instructions

1. Each task must be completed individually.
2. Programs must be implemented using C++.
3. All programs should be compiled and tested.
4. The GitHub repository must contain Task01 to Task12 folders.

5. Each task folder should include source code, input files and output screenshots.
6. A README file should explain the implementation.
7. UML diagrams, pseudocode and test tables should be included wherever required.
8. Progressive commits should be maintained in the GitHub repository.

TASK 1 – UNDERSTANDING THE PROBLEM AND OOP CONCEPTS **4 Marks**

Question

Explain how Object-Oriented Programming can be used to design the metro smart-card fare system. Discuss procedural programming versus object-oriented programming, the main OOP principles, journey rules and alternative design complexity.

Answer

1. Introduction to Object-Oriented Programming

Object-Oriented Programming (OOP) is a programming approach that organizes a program around objects and classes. An object represents a real-world entity and contains both data and functions.

In the metro smart-card system, a passenger's smart card can be represented as an object. It contains information such as card balance and provides functions to calculate whether a journey is permitted and deduct the fare.

OOP helps to organize the program into separate modules, making it easier to understand, maintain, test and extend.

2. Procedural Programming

Procedural programming is a programming approach in which a program is divided into functions. Data and functions are generally maintained separately.

For example, the metro system can use separate functions such as:

- calculateFare() – calculates the journey fare.
- checkBalance() – checks the available balance.
- deductFare() – deducts the fare.
- displayBalance() – displays the remaining balance.

Advantages:

1. Easy to understand for small programs.
2. Functions can be reused.
3. Simple to implement.
4. Requires fewer programming concepts.

Disadvantages:

1. Data may be accessed or modified by different functions.
2. Maintaining large programs becomes difficult.
3. Security of data is limited.
4. New features may require changes in many functions.

3. Object-Oriented Programming

In OOP, data and functions are combined into classes. Objects are created from these classes.

For the metro system, the following classes can be designed:

- SmartCard – stores passenger balance and manages deductions.
- FareTable – stores base fares for different zones.
- Journey – stores journey information.
- Passenger – stores passenger details and concession information.

Advantages:

1. Provides data security through encapsulation.
2. Supports code reuse through inheritance.
3. Supports flexibility through polymorphism.
4. Makes the system easier to maintain.
5. Supports future expansion.
6. Reduces duplication of code.

4. Main Principles of OOP

a) Encapsulation

Encapsulation means combining data and functions into a single unit called a class. It also restricts direct access to important data.

Example: The smart-card balance is declared as private. It can only be changed using authorized member functions.

b) Abstraction

Abstraction means hiding unnecessary implementation details and showing only essential operations.

Example: The passenger uses a deductFare() function without knowing the internal calculation process.

c) Inheritance

Inheritance allows one class to acquire the properties and functions of another class.

Example: StudentPassenger and SeniorPassenger can inherit common passenger information from a Passenger class.

d) Polymorphism

Polymorphism allows the same function or interface to behave differently in different situations.

Example: A concession calculation function can calculate different discounts for different passenger categories.

5. Metro Journey Rules

The following rules are considered in the system:

1. The passenger selects one of the three travel zones.
2. Each zone has a predefined base fare.
3. A 20% surcharge is applied during peak hours.
4. Morning peak hours are 8:00 AM to before 10:00 AM.
5. Evening peak hours are 5:00 PM to before 8:00 PM.
6. The remaining balance must be at least ₹50 after the fare is deducted.
7. Invalid zone numbers and invalid time values must be rejected.
8. Fare calculations should use a suitable numeric representation to avoid rounding errors.

6. Procedural Programming versus OOP

Feature	Procedural Programming	Object-Oriented Programming
Main unit	Function	Class and object
Data handling	Separate from functions	Combined with functions
Data security	Limited	Encapsulation
Reusability	Function reuse	Inheritance and composition
Maintenance	Difficult for large systems	Easier for modular systems
Expansion	Changes may affect many functions	New classes and methods can be added
Example	deductFare() function	SmartCard class

7. Alternative Concept: Inheritance-Based Passenger Design

An alternative design is to create a base class called Passenger and derive StudentPassenger and SeniorPassenger classes from it.

The base class can contain common passenger information, while derived classes can implement different concession rules.

This approach avoids writing separate passenger information code repeatedly.

Complexity Analysis:

- Creating a passenger object: O(1).
- Accessing passenger information: O(1).
- Calculating one journey fare: O(1).
- Processing n journeys: O(n).

Conclusion

Object-Oriented Programming is more suitable for the metro smart-card system because it provides data protection, modularity, code reuse and easy maintenance. Encapsulation is especially important because it prevents unauthorized changes to the smart-card balance.

TASK 2 – C++ TOKENS, KEYWORDS, IDENTIFIERS AND CONSTANTS 3 Marks

Question

Explain tokens, keywords, identifiers and constants in C++. Compare #define, const and constexpr. Discuss enum and enum class, naming conventions and alternative design complexity.

Answer

1. Tokens in C++

Tokens are the smallest individual units of a C++ program recognized by the compiler.

The main types of tokens are:

1. Keywords.
2. Identifiers.
3. Constants.
4. String literals.
5. Operators.
6. Punctuators.

Example:

```
int fare = 1000;
```

Here:

- int is a keyword.
- fare is an identifier.
- 1000 is an integer constant.

- = is an assignment operator.
- ; is a punctuator.

2. Keywords

Keywords are reserved words in C++ that have predefined meanings. They cannot be used as variable names.
Examples:
int, float, double, class, public, private, if, else, for, while, return, const, and switch.
In the metro system, keywords such as class, private, public and if are used to create classes and implement fare conditions.

3. Identifiers

Identifiers are names given to variables, functions, classes and objects.
Examples:

```
int balance;
int baseFare;
class SmartCard;
void calculateFare();
```

Rules for identifiers:

1. An identifier can contain letters, digits and underscores.
2. It must not begin with a digit.
3. Spaces are not allowed.
4. C++ keywords cannot be used.
5. Identifiers are case-sensitive.
6. Meaningful names should be used.

4. Constants

Constants are fixed values that cannot be changed during program execution.
Example:

```
const int MIN_BALANCE = 5000;
constexpr int PEAK_PERCENT = 20;
```

The metro fare system uses constants for base fares, surcharge percentages and minimum balance requirements.

5. Difference Between #define, const and constexpr

Feature	#define	const	constexpr
Type checking	No	Yes	Yes
Processing	Preprocessor	Compiler	Compiler
Scope	Depends on preprocessing location	Normal C++ scope	Normal C++ scope
Value modification	Text substitution	Not allowed	Not allowed
Compile-time value	Not necessarily	Not necessarily	Required when initialized correctly

Example:

```
#include <iostream>
using namespace std;

#define BASE_FARE 1000

const int MIN_BALANCE = 5000;
constexpr int PEAK_PERCENT = 20;

int main()
{
   cout << BASE_FARE << endl;
   cout << MIN_BALANCE << endl;
   cout << PEAK_PERCENT << endl;

   return 0;
}
```

Output:

```
1000
5000
20
```

Explanation: #define performs text substitution, while const and constexpr are C++ declarations with type information. constexpr is useful when a value must be available during compilation.

6. Difference Between enum and enum class

An enumeration defines a group of named constant values.

Example using enum:

```
enum Zone
{
   Inner,
   Middle,
   Outer
};
```

Example using enum class:

```
enum class Zone
{
   Inner,
   Middle,
   Outer
};
```

Feature	**enum**	**enum class**
Scope	Enumerator names are available in surrounding scope	Enumerator names are scoped
Implicit conversion to integer	Allowed	Not allowed
Type safety	Lower	Higher
Name conflicts	More likely	Less likely

For the metro system, enum class is recommended because it prevents accidental mixing of zone values with unrelated integer values.

Example:

```
#include <iostream>
using namespace std;

enum class Zone
{
   Inner,
   Middle,
   Outer
};

int main()
{
   Zone z = Zone::Inner;

   if (z == Zone::Inner)
      cout << "Inner Zone";

   return 0;
}
```

Output:

```
Inner Zone
```

7. Naming Conventions

Meaningful names improve readability and maintenance.

Recommended names:

- baseFare
- minimumBalance
- peakSurcharge
- calculateFare
- SmartCard
- Journey

Constants should use clear uppercase names such as MIN_BALANCE_PAISE and PEAK_SURCHARGE_PERCENT.

8. Alternative Concept: Using a Macro for Fare Values

An alternative is to define fare values using macros.

```
#define INNER_FARE 1000
#define MIDDLE_FARE 2000
#define OUTER_FARE 3000
```

This works for simple substitution but does not provide normal type checking or scope rules.

Complexity Analysis:

- Accessing a constant: O(1).
- Comparing an enum value: O(1).
- Selecting one fare using a switch: O(1) for a fixed number of zones.
- Additional memory for compile-time constants: generally no separate runtime storage is required when optimized.

Conclusion

C++ tokens and constants form the foundation of the fare program. Using constexpr for fixed values and enum class for zones improves type safety, readability and maintainability.

TASK 3 – DATA TYPES AND MEMORY REPRESENTATION **4 Marks**

Question

Explain the different data types in C++. Discuss the sizeof operator, data type ranges, the representation of money using integer paise instead of double, type compatibility, overflow and alternative complexity.

Answer

1. Introduction to Data Types

A data type specifies the kind of value a variable can store. It also determines the memory required and the operations that can be performed on that variable.

C++ provides different data types to store integers, decimal numbers, characters and logical values.

2. Common C++ Data Types

Data type	Purpose	Example
int	Stores integer values	int zone = 2;
short	Stores smaller integer values	short count = 10;
long long	Stores large integer values	long long total = 5000000;
float	Stores decimal values	float fare = 10.5f;
double	Stores decimal values with higher precision	double amount = 20.75;
char	Stores a character	char grade = 'A';
bool	Stores true or false	bool peak = true;

3. The sizeof Operator

The sizeof operator returns the memory size of a data type or variable in bytes.

Example:

```
#include <iostream>
using namespace std;

int main()
{
   cout << "Size of int: " << sizeof(int) << " bytes" << endl;
   cout << "Size of float: " << sizeof(float) << " bytes" << endl;
   cout << "Size of double: " << sizeof(double) << " bytes" << endl;
   cout << "Size of char: " << sizeof(char) << " byte" << endl;
   cout << "Size of long long: " << sizeof(long long) << " bytes" << endl;

   return 0;
}
```

Typical output on many modern systems:

```
Size of int: 4 bytes
Size of float: 4 bytes
Size of double: 8 bytes
Size of char: 1 byte
Size of long long: 8 bytes
```

The exact size of some types is implementation-dependent. The values above are typical, not universal.

4. Integer Ranges

The range of an integer type depends on its implementation and number of value bits. On common systems:

Data type	Typical size	Typical range
short	2 bytes	-32,768 to 32,767
int	4 bytes	-2,147,483,648 to 2,147,483,647
long long	8 bytes	Approximately -9.22×10^{18} to 9.22×10^{18}

5. Representing Money Using Integer Paise

Money should be represented carefully because floating-point values may not store every decimal fraction exactly.

For example, a fare of ₹10.50 can be represented as 1050 paise.

Instead of:

```
double fare = 10.50;
```

Use:

```
long long farePaise = 1050;
```

The program performs calculations using integer paise and converts the amount into rupees only when displaying it.

Example:

```
#include <iostream>
using namespace std;

int main()
{
   long long farePaise = 1050;

   cout << "Fare: Rs "
        << farePaise / 100 << "."
        << (farePaise % 100 < 10 ? "0" : "")
        << farePaise % 100 << endl;

   return 0;
}
```

Output:

```
Fare: Rs 10.50
```

Advantages of integer paise:

1. Avoids many floating-point rounding errors.
2. Provides exact representation of whole paise.
3. Makes fare comparisons reliable.
4. Supports accurate balance deductions.
5. Simplifies financial calculations.

6. Type Compatibility

Type compatibility determines whether values of one data type can be assigned or converted to another data type.

Example:

```
int a = 10;
double b = a;
```

Here, the integer value is converted into a double value.

However, converting a decimal value into an integer may remove its fractional part.

```
double fare = 10.75;
int amount = fare;

cout << amount;
```

Output:

```
10
```

The fractional part is lost. Therefore, the program must select appropriate data types for fare calculations.

7. Overflow

Overflow occurs when a calculation produces a value outside the range that a data type can represent.

For example, a signed 32-bit integer cannot represent values greater than 2,147,483,647.

The total fare for a large metro network may exceed this limit.

If the system processes millions of journeys, using int for the total amount can be unsafe.

Using long long provides a much larger range.

8. Alternative Concept: Using double for Money

An alternative is to store fares in rupees using double.

```
#include <iostream>
using namespace std;

int main()
{
   double fare = 10.50;
   double total = fare * 3;

   cout << "Total fare: Rs " << total << endl;

   return 0;
}
```

Output:

Total fare: Rs 31.5

For display, fixed and setprecision(2) can be used to show two decimal places.

```
#include <iomanip>
cout << fixed << setprecision(2) << total;
```

Complexity Analysis:

- Storing one fare: O(1).
- Adding two fare values: O(1).
- Processing n journeys: O(n).
- One long long variable typically uses 8 bytes.

Conclusion

For a metro fare system, long long integer paise is a suitable choice because it supports exact paise calculations and large network totals. double can be used for display or non-financial calculations, but integer paise is preferable for storing and calculating fare amounts.

TASK 4 – TYPE CONVERSIONS AND EXPRESSIONS **4 Marks**

Question

Explain implicit conversions, integer division, signed and unsigned values, narrowing conversions and static_cast. Predict the output of a given program and discuss an alternative approach.

Answer

1. Type Conversion in C++

Type conversion is the process of converting a value from one data type to another.

There are two major types:

1. Implicit type conversion.
2. Explicit type conversion.

2. Implicit Type Conversion

Implicit conversion is automatically performed by the compiler when a value is assigned or used with another compatible type.

Example:

```
#include <iostream>
using namespace std;

int main()
{
   int amount = 100;
   double total = amount;

   cout << total << endl;

   return 0;
}
```

Output:

100

Here, the integer value is automatically converted into a double value.

3. Integer Division

When two integer values are divided, the fractional part is removed.

Example:

```
#include <iostream>
using namespace std;

int main()
{
   int a = 5;
   int b = 2;

   cout << a / b << endl;

   return 0;
}
```

Output:

2

The mathematical result is 2.5, but integer division produces 2.

To obtain a decimal result, at least one operand must be converted to a floating-point type.

```
cout << static_cast<double>(a) / b;
```

Output:

2.5

4. Signed and Unsigned Data Types

A signed data type can represent positive and negative values.

An unsigned data type represents only non-negative values.

Example:

```
int balance = -100;
unsigned int count = 100;
```

For the metro system, signed integer types are useful when calculations may temporarily involve negative differences. Unsigned values must be used carefully because subtracting a larger value from a smaller unsigned value can produce a very large positive result.

5. Narrowing Conversion

Narrowing occurs when a value is converted into a type that cannot represent the original value or its precision.

Example:

```
double fare = 25.75;
int amount = fare;

cout << amount;
```

Output:

25

The decimal part is removed.

Narrowing conversions should be avoided when they can cause loss of important information.

6. Explicit Conversion Using static_cast

static_cast is used to perform an explicit type conversion.

Example:

```
#include <iostream>
using namespace std;

int main()
{
   int totalPaise = 1250;

   double rupees = static_cast<double>(totalPaise) / 100;

   cout << rupees << endl;

   return 0;
}
```

Output:

12.5

Here, static_cast<double> converts the integer value into a double before division.

7. Predicting Program Output

Consider the following example:

```
#include <iostream>
using namespace std;

int main()
{
    int a = 10;
    int b = 3;

    cout << a / b << endl;
    cout << static_cast<double>(a) / b << endl;

    double x = 10.75;
    int y = x;

    cout << y << endl;

    return 0;
}
```

Output:

```
3
3.33333
10
```

Explanation:

- 10 / 3 performs integer division and produces 3.
- static_cast<double>(10) / 3 produces a decimal result.
- Assigning 10.75 to an integer removes the fractional part and produces 10.

8. Application in Metro Fare Calculation

The metro fare system should avoid unintended conversions.

For example, a 20% surcharge can be calculated using integer arithmetic:

```
long long fare = 1000;
long long surcharge = fare * 20 / 100;
long long finalFare = fare + surcharge;
```

Output:

```
Final fare in paise: 1200
```

This represents ₹12.00.

9. Alternative Concept: Floating-Point Division

Instead of using integer arithmetic for every calculation, the program can use double for percentage calculations.

Example:

```
double fare = 10.00;
double finalFare = fare + (fare * 20.0 / 100.0);
```

Output:

```
12
```

This approach is simple, but financial calculations should be carefully rounded before storing or charging amounts.

Complexity Analysis:

- Type conversion: O(1).
- Division: O(1).
- Fare calculation: O(1).
- Processing n journeys: O(n).

Conclusion

Understanding type conversion is important for preventing incorrect fare values. Integer division, narrowing and signed/unsigned conversions can cause errors. Using static_cast for intentional conversions and integer paise for money makes the system safer.

TASK 5 – OPERATORS AND EXPRESSIONS IN C++ **4 Marks**

Question

Explain expressions and operator precedence, increment and decrement operators, negative division and modulus operations. Predict the output of a program and discuss lookup-table implementation as an alternative.

Answer

1. Expressions in C++

An expression is a combination of variables, constants and operators that produces a value.
Example:
int total = 100 + 50;
Here, 100 + 50 is an expression.
Expressions are used in the metro system to calculate fares, surcharges and remaining balance

2. Operator Precedence

Operator precedence determines the order in which operators are evaluated.
For example:
int result = 10 + 5 * 2;
Output:
20
Multiplication is performed before addition.
Parentheses can be used to control the order.
int result = (10 + 5) * 2;
Output:
30

3. Increment and Decrement Operators

The increment operator ++ increases a variable by one.
The decrement operator -- decreases a variable by one.
There are two forms:
Pre-increment:
int a = 5;
cout << ++a;
Output:
6
The value is increased before it is used.
Post-increment:
int a = 5;
cout << a++;
Output:
5
The original value is used first, and then it is increased.

4. Division and Modulus with Negative Values

The division operator / performs division.
The modulus operator % returns the remainder for integer operands.
Example:

```
#include <iostream>
using namespace std;

int main()
{
   cout << 7 / 3 << endl;
   cout << 7 % 3 << endl;
   cout << -7 / 3 << endl;
   cout << -7 % 3 << endl;

   return 0;
}
```

Output:
2
1
-2
-1
In C++, integer division truncates toward zero. The remainder has the sign of the dividend when it is non-zero.

5. Application to Peak-Hour Calculation

Peak hours can be checked by converting the time into minutes since midnight.

```
bool isPeak(int hour, int minute)
{
   int totalMinutes = hour * 60 + minute;

   return (totalMinutes >= 480 && totalMinutes < 600) ||
          (totalMinutes >= 1020 && totalMinutes < 1200);
}
```

The intervals are:

- 08:00 to before 10:00.
- 17:00 to before 20:00.

This correctly handles the boundaries.

Time	Peak status
07:59	Off-peak
08:00	Peak
09:59	Peak
10:00	Off-peak
16:59	Off-peak
17:00	Peak
19:59	Peak
20:00	Off-peak

6. Example Program

```
#include <iostream>
using namespace std;

bool isPeak(int hour, int minute)
{
   int t = hour * 60 + minute;

   return (t >= 480 && t < 600) ||
          (t >= 1020 && t < 1200);
}

int main()
{
   int hour = 9;
   int minute = 30;

   if (isPeak(hour, minute))
      cout << "Peak Hour";
   else
      cout << "Off-Peak Hour";

   return 0;
}
```

Output:

```
Peak Hour
```

7. Alternative Concept: Lookup Table

Instead of using multiple conditional expressions, the program can store fare values in an array and access them using a zone index.

Example:

```
#include <iostream>
using namespace std;

int main()
{
```

```
    int baseFare[] = {1000, 2000, 3000};
    int zone = 1;

    cout << "Base fare: " << baseFare[zone] << " paise";

    return 0;
}
```

Output:
Base fare: 2000 paise
The array stores the fares for Inner, Middle and Outer zones.
Complexity Analysis:

- Arithmetic operation: O(1).
- Peak-time checking: O(1).
- Lookup-table access: O(1).
- Space for three fare values: O(1).

Conclusion
Operators and expressions are essential for fare calculation. Correct operator precedence and proper handling of negative division are important. Using minute-based peak-hour checking avoids errors at 10:00 AM and 8:00 PM. A lookup table also makes fare management easier.

TASK 6 – CONTROL STRUCTURES AND FARE PROGRAM IMPLEMENTATION 4 Marks

Question

Explain control structures, loops, switch statements, break and continue. Write pseudocode for the metro fare system and implement the required C++ code using switch statements and integer paise. Discuss an array lookup alternative.

Answer

1. Introduction to Control Structures

Control structures determine the order in which statements are executed in a program.
They are used to make decisions, repeat operations and control the flow of execution.
The main types are:

1. Sequential control structure.
2. Selection control structure.
3. Iteration control structure.

2. Sequential Control Structure

In sequential execution, statements are executed one after another in the order in which they appear.
Example:

```
int fare = 1000;
int surcharge = 200;
int total = fare + surcharge;
```

The statements are executed sequentially.

3. Selection Control Structure

Selection statements execute different code depending on a condition.
Examples:

- if
- if-else
- else-if
- switch

Example:

```
if (balance >= 5000)
    cout << "Travel allowed";
else
    cout << "Insufficient balance";
```

4. Iteration Control Structure

Iteration statements repeat a block of code.
Examples:

- for
- while
- do-while

A for loop can process multiple passenger journeys.

```
for (int i = 1; i <= 5; i++)
{
```

```
    cout << "Journey " << i << endl;
}
```

5. Switch Statement

The switch statement selects one case based on the value of an expression.

Example:

```
switch (zone)
{
    case 0:
        cout << "Inner Zone";
        break;

    case 1:
        cout << "Middle Zone";
        break;

    case 2:
        cout << "Outer Zone";
        break;

    default:
        cout << "Invalid Zone";
}
```

The break statement terminates the current switch case and prevents execution from continuing into the next case.

6. Break and Continue

Break: Terminates the nearest loop or switch statement.

Continue: Skips the remaining statements in the current loop iteration and proceeds to the next iteration.

Example:

```
for (int i = 1; i <= 5; i++)
{
    if (i == 3)
        continue;

    cout << i << " ";
}
```

Output:

```
1 2 4 5
```

7. Algorithm for Metro Fare Calculation

Step 1: Start the program.
Step 2: Declare the base fares for Inner, Middle and Outer zones.
Step 3: Read the passenger's zone, hour and minute.
Step 4: Validate the zone and time values.
Step 5: Use a switch statement to select the base fare.
Step 6: Convert the time into minutes since midnight.
Step 7: Check whether the journey is during peak hours.
Step 8: If it is peak time, calculate the 20% surcharge.
Step 9: Calculate the final fare in paise.
Step 10: Check whether the remaining balance is at least ₹50.
Step 11: If the condition is satisfied, deduct the fare.
Step 12: Otherwise, reject the journey.
Step 13: Display the fare and remaining balance.
Step 14: Stop the program.

8. Pseudocode

```
START

SET INNER_FARE = 1000
SET MIDDLE_FARE = 2000
SET OUTER_FARE = 3000
SET MINIMUM_BALANCE = 5000
```

```
READ zone
READ hour
READ minute
READ balance

IF zone is invalid OR time is invalid
   DISPLAY "Invalid Input"
   STOP
END IF

SWITCH zone
   CASE 0:
      fare = INNER_FARE
   CASE 1:
      fare = MIDDLE_FARE
   CASE 2:
      fare = OUTER_FARE
   DEFAULT:
      DISPLAY "Invalid Zone"
      STOP
END SWITCH

time = hour * 60 + minute

IF time is between 480 and 599
   OR time is between 1020 and 1199
   surcharge = fare * 20 / 100
   fare = fare + surcharge
END IF

IF balance - fare >= MINIMUM_BALANCE
   balance = balance - fare
   DISPLAY "Journey Approved"
   DISPLAY fare
   DISPLAY balance
ELSE
   DISPLAY "Insufficient Balance"
END IF

STOP
```

9. C++ Implementation

```
#include <iostream>
using namespace std;

int main()
{
   const long long INNER_FARE = 1000;
   const long long MIDDLE_FARE = 2000;
   const long long OUTER_FARE = 3000;
   const long long MIN_BALANCE = 5000;

   int zone, hour, minute;
   long long balance = 10000;
   long long fare = 0;

   cout << "Enter zone (0=Inner, 1=Middle, 2=Outer): ";
   cin >> zone;

   cout << "Enter hour and minute: ";
   cin >> hour >> minute;
```

```
    if (zone < 0 || zone > 2 ||
        hour < 0 || hour > 23 ||
        minute < 0 || minute > 59)
    {
        cout << "Invalid Input";
        return 0;
    }

    switch (zone)
    {
        case 0:
            fare = INNER_FARE;
            break;

        case 1:
            fare = MIDDLE_FARE;
            break;

        case 2:
            fare = OUTER_FARE;
            break;

        default:
            cout << "Invalid Zone";
            return 0;
    }

    int totalMinutes = hour * 60 + minute;

    bool peak =
        (totalMinutes >= 480 && totalMinutes < 600) ||
        (totalMinutes >= 1020 && totalMinutes < 1200);

    if (peak)
    {
        fare = fare + (fare * 20 / 100);
    }

    cout << "Calculated fare: " << fare << " paise" << endl;

    if (balance - fare >= MIN_BALANCE)
    {
        balance = balance - fare;

        cout << "Journey Approved" << endl;
        cout << "Remaining balance: " << balance << " paise" << endl;
    }
    else
    {
        cout << "Journey Denied" << endl;
        cout << "Insufficient Balance" << endl;
    }

    return 0;
}
```

10. Sample Execution

Input:

Enter zone (0=Inner, 1=Middle, 2=Outer): 1

Enter hour and minute: 9 30

Output:
Calculated fare: 2400 paise
Journey Approved
Remaining balance: 7600 paise
Explanation:

- Zone selected: Middle.
- Base fare: ₹20.
- Time: 09:30 AM.
- Peak surcharge: 20% of ₹20 = ₹4.
- Final fare: ₹24.
- Initial balance: ₹100.
- Remaining balance: ₹76.

11. Alternative Concept: Array Lookup

Instead of using a switch statement, an array can store the base fare values.

```
#include <iostream>
using namespace std;

int main()
{
   const long long fares[] = {1000, 2000, 3000};

   int zone = 2;

   if (zone >= 0 && zone < 3)
      cout << "Fare: " << fares[zone] << " paise";
   else
      cout << "Invalid Zone";

   return 0;
}
```

Output:
Fare: 3000 paise
Complexity Analysis:

- Selecting a fare using switch: O(1) for the fixed three-zone selection.
- Selecting a fare using an array: O(1).
- Checking peak time: O(1).
- Processing one journey: O(1).
- Processing n journeys: O(n).

Conclusion
Control structures are necessary for implementing the metro fare system. The switch statement selects the appropriate zone fare, conditional statements apply peak-hour surcharges, and loops support repeated journey processing. Integer paise helps maintain accurate fare calculations.

TASK 7 – FROM PROCEDURAL PROGRAMMING TO OBJECT-ORIENTED PROGRAMMING

4 Marks

Question

Rewrite the fare logic using procedural and object-oriented programming. Discuss separate variables and free functions, the SmartCard class with private balance, deduct() and canTravel() functions, prevention of negative balance, and which approach is easier to modify. Also discuss the alternative of using a public balance member.

Answer

1. Procedural Approach

In procedural programming, data and functions are maintained separately. The smart-card balance can be stored in a global variable, and separate functions can calculate the fare and deduct the amount.
Example:

```
#include <iostream>
using namespace std;

long long balance = 10000;
const long long MIN_BALANCE = 5000;

bool canTravel(long long fare)
```

```
{
   return balance - fare >= MIN_BALANCE;
}

void deduct(long long fare)
{
   balance = balance - fare;
}

int main()
{
   long long fare = 2000;

   if (canTravel(fare))
   {
      deduct(fare);
      cout << "Journey Approved" << endl;
      cout << "Balance: " << balance << " paise";
   }
   else
   {
      cout << "Journey Denied";
   }

   return 0;
}
```

Output:
Journey Approved
Balance: 8000 paise

Advantages:

1. Simple to write and understand.
2. Suitable for small programs.
3. Functions can be reused.
4. Requires fewer class definitions.

Disadvantages:

1. Global data can be modified by different functions.
2. Any function can accidentally change the balance.
3. The minimum-balance rule depends on every caller following the correct procedure.
4. Maintenance becomes difficult as the system grows.

2. Object-Oriented Approach

In object-oriented programming, the balance and its related operations are combined inside the SmartCard class. The balance is declared as private, so it cannot be directly modified by outside code.

Example:

```
#include <iostream>
using namespace std;

class SmartCard
{
private:
   long long balance;
   const long long MIN_BALANCE = 5000;

public:
   SmartCard(long long initialBalance)
   {
      balance = initialBalance;
   }

   bool canTravel(long long fare) const
   {
```

```
        return fare >= 0 &&
               balance - fare >= MIN_BALANCE;
    }

    bool deduct(long long fare)
    {
        if (!canTravel(fare))
            return false;

        balance -= fare;
        return true;
    }

    long long getBalance() const
    {
        return balance;
    }
};

int main()
{
    SmartCard card(10000);

    long long fare = 2000;

    if (card.deduct(fare))
        cout << "Journey Approved" << endl;
    else
        cout << "Journey Denied" << endl;

    cout << "Remaining Balance: "
         << card.getBalance() << " paise";

    return 0;
}
```

Output:

Journey Approved

Remaining Balance: 8000 paise

3. How the Class Prevents a Negative Balance

The SmartCard class protects the balance using encapsulation.

1. The balance is declared as a private data member.
2. Outside functions cannot directly modify it.
3. The canTravel() function checks whether the passenger has enough balance.
4. The deduct() function performs the check before deducting the fare.
5. If the balance is insufficient, the function returns false without changing the balance.

The constructor should also validate the initial balance in a production-quality implementation. Recharge functions should reject zero or negative recharge amounts.

4. Procedural versus Object-Oriented Comparison

Feature	Procedural	Object-oriented
Balance storage	Global variable	Private class member
Data protection	Limited	Encapsulation
Fare deduction	Separate function	Class member function
Minimum balance	Must be checked by caller	Checked inside deduct()
Maintenance	Difficult for large systems	Easier
Reusability	Function-based	Class-based

5. Which Approach Is Easier to Change?

The object-oriented approach is easier to modify because the minimum-balance rule is maintained inside one class.

If the minimum balance changes from ₹50 to ₹75, the class constant can be updated in one place. The internal deduction logic remains protected.
In a procedural design, every function that changes the balance must be checked to ensure that it follows the new rule.

6. Alternative Concept: Public Balance Member

An alternative is to declare the balance as a public member.

```
#include <iostream>
using namespace std;

struct SmartCard
{
   long long balance;
};

int main()
{
   SmartCard card{10000};

   card.balance = -500;

   cout << "Balance: " << card.balance << " paise";

   return 0;
}
```

Output:
Balance: -500 paise
This program allows the balance to become negative because any part of the program can modify it directly.
Complexity Analysis:

- Balance access: O(1).
- Balance deduction: O(1).
- Balance validation: O(1).
- Processing n journeys: O(n).
- Space for one balance variable: typically 8 bytes using long long.

Conclusion
The object-oriented approach is recommended because it protects the smart-card balance and ensures that the minimum-balance rule is checked before every deduction. A private data member with controlled member functions is safer than a public balance variable.

TASK 8 – SAFE INPUT AND FORMATTED OUTPUT **4 Marks**

Question

Design safe input and output for the metro gate program. Explain cin, cin.fail(), cin.clear(), cin.ignore(), formatted output using <iomanip> and journey reports. Compare cin with getline() and stoi().

Answer

1. Introduction

Input validation is an important part of a reliable metro fare system. Passengers or operators may enter invalid values such as letters instead of numbers, incorrect zone numbers or invalid time values.
If the program does not validate input, it may calculate an incorrect fare or stop processing further journeys.

2. Reading Input Using cin

The cin object is used to read values from the keyboard.
Example:

```
int zone;

cout << "Enter zone: ";
cin >> zone;
```

If the user enters a valid integer, the value is stored in the variable.
However, if the user enters letters such as abc, the input operation fails.

3. Checking cin.fail()

The cin.fail() function checks whether the previous input operation failed.
Example:

```
int zone;
```

```
cin >> zone;

if (cin.fail())
{
   cout << "Invalid input";
}
```
If a user enters letters when an integer is expected, the stream enters a failed state.

4. Using cin.clear() and cin.ignore()

Two operations are required to recover from invalid input.

- cin.clear() resets the error state.
- cin.ignore() removes unwanted characters from the input buffer.

Example:

```
#include <iostream>
#include <limits>
using namespace std;

int main()
{
   int zone;

   cout << "Enter zone: ";

   while (!(cin >> zone))
   {
      cout << "Invalid input. Enter again: ";

      cin.clear();

      cin.ignore(
         numeric_limits<streamsize>::max(),
         '\n'
      );
   }

   cout << "Selected zone: " << zone;

   return 0;
}
```
Sample input:
Enter zone: abc
Invalid input. Enter again: 2
Output:
Selected zone: 2

5. Formatted Output Using <iomanip>

The <iomanip> header provides formatting tools.
Important functions include:

- setw() – sets the minimum width of the next output field.
- setfill() – sets the character used to fill empty positions.
- fixed – uses fixed-point decimal notation.
- setprecision() – controls decimal precision.

Example:

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
   double fare = 12.5;

   cout << fixed << setprecision(2);
```

```
    cout << "Fare: Rs " << fare << endl;

    return 0;
}
```

Output:

```
Fare: Rs 12.50
```

6. Journey Report Program

The following program reads a journey count, validates each journey and displays a formatted report.

```
#include <iostream>
#include <iomanip>
#include <limits>
using namespace std;

int main()
{
    const long long BASE[] = {1000, 2000, 3000};
    const long long MIN_BALANCE = 5000;

    long long balance = 10000;
    int n;

    cout << "Enter number of journeys: ";

    while (!(cin >> n) || n < 0)
    {
        cout << "Invalid count. Enter again: ";

        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }

    cout << left << setw(8) << "No"
         << setw(10) << "Zone"
         << setw(10) << "Time"
         << setw(12) << "Fare"
         << "Balance" << endl;

    for (int i = 1; i <= n; i++)
    {
        int zone, hour, minute;

        cout << "Enter zone, hour and minute: ";

        if (!(cin >> zone >> hour >> minute))
        {
            cout << "Invalid input. Journey skipped." << endl;

            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            continue;
        }

        if (zone < 0 || zone > 2 ||
            hour < 0 || hour > 23 ||
            minute < 0 || minute > 59)
        {
            cout << "Invalid journey. Skipped." << endl;
            continue;
        }
```

```
        long long fare = BASE[zone];

        int t = hour * 60 + minute;

        bool peak =
           (t >= 480 && t < 600) ||
           (t >= 1020 && t < 1200);

        if (peak)
           fare += fare * 20 / 100;

        if (balance - fare >= MIN_BALANCE)
        {
           balance -= fare;

           cout << left << setw(8) << i
                << setw(10) << zone
                << setw(2) << setfill('0') << hour << ":"
                << setw(2) << minute << setfill(' ')
                << setw(12) << fixed << setprecision(2)
                << fare / 100.0
                << balance / 100.0 << endl;
        }
        else
        {
           cout << "Journey denied due to low balance." << endl;
        }
     }

     return 0;
}
```

7. Alternative Concept: getline() and stoi()

Another approach is to read the complete input line using getline() and convert it into an integer using stoi().
Example:

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
   string line;

   cout << "Enter a number: ";
   getline(cin, line);

   try
   {
      size_t pos;
      int value = stoi(line, &pos);

      if (pos != line.size())
      {
         cout << "Invalid input";
         return 0;
      }

      cout << "Value: " << value;
   }
   catch (const exception&)
   {
```

```
        cout << "Invalid input";
    }

    return 0;
}
```

Input:
Enter a number: abc
Output:
Invalid input
Comparison:

Feature	cin	getline() with stoi()
Input method	Extracts typed values	Reads a full line
Invalid input	Sets stream failure state	Conversion may throw an exception
Recovery	Requires clear() and ignore()	Can reject one line and continue
Memory	No full-line string required	Stores the input line
Best use	Simple numeric input	Line-based records

Complexity Analysis:
For an input line of length L:

- Reading and examining the line: O(L).
- Converting the text to an integer: O(L).
- Temporary storage for the line: O(L).

Conclusion
Safe input validation prevents incorrect fare calculations and program failure. cin.fail(), cin.clear() and cin.ignore() are useful for recovering from invalid input. Formatted output makes the journey report easier to read. getline() and stoi() are useful when complete input lines must be validated.

TASK 9 – IMPLEMENT AND TEST THE FARE LOGIC **4 Marks**

Question

Implement and test the fare logic. Discuss normal journeys, peak and off-peak journeys, boundary cases, invalid cases, a five-journey calculation and an alternative table-driven testing approach.

Answer

1. Introduction

Testing is the process of checking whether a program produces the expected output for different inputs.
The metro fare system must be tested using normal, boundary and invalid inputs. Testing helps identify errors in fare calculation, peak-hour detection and balance deduction.

2. Normal Journey Test Cases

Test case	Zone	Time	Expected fare
Inner off-peak	Inner	12:00	₹10
Middle off-peak	Middle	12:00	₹20
Outer off-peak	Outer	12:00	₹30
Inner peak	Inner	09:00	₹12
Middle peak	Middle	18:00	₹24
Outer peak	Outer	08:30	₹36

3. Boundary Test Cases

The system must correctly handle the beginning and ending of the peak periods.

Time Expected status

07:59 Off-peak

08:00 Peak

09:59 Peak

10:00 Off-peak

16:59 Off-peak

17:00 Peak

19:59 Peak

20:00 Off-peak

The correct peak-hour condition is:

```
bool peak =
    (t >= 480 && t < 600) ||
    (t >= 1020 && t < 1200);
```

This uses half-open intervals. The starting time is included, while the ending time is excluded.

4. Minimum Balance Test Cases

The minimum balance after fare deduction must be at least ₹50.

Initial balance	Fare	Remaining balance	Result
₹60	₹10	₹50	Allowed
₹59.99	₹10	₹49.99	Denied
₹60	₹10.01	₹49.99	Denied
₹100	₹24	₹76	Allowed

5. Invalid Input Test Cases

Input	Expected result
Zone = 5	Invalid zone
Zone = -1	Invalid zone
Hour = 25	Invalid time
Minute = 70	Invalid time
Hour = abc	Invalid numeric input
Recharge = -200 paise	Reject recharge

6. Five-Journey Manual Calculation

Consider the following five journeys:

Journey	Zone	Time	Fare calculation	Fare
1	Inner	12:00	Off-peak	₹10
2	Middle	09:00	₹20 + 20%	₹24
3	Outer	18:30	₹30 + 20%	₹36
4	Inner	10:00	Off-peak	₹10
5	Outer	20:00	Off-peak	₹30

Total fare:

$$10+24+36+10+30=₹110$$

Therefore, the total fare for five journeys is ₹110, or 11,000 paise.

7. C++ Test Program

```
#include <iostream>
using namespace std;

long long fare(int zone, int hour, int minute)
{
    const long long BASE[] = {1000, 2000, 3000};

    if (zone < 0 || zone > 2 ||
        hour < 0 || hour > 23 ||
        minute < 0 || minute > 59)
    {
        return -1;
    }

    long long amount = BASE[zone];

    int t = hour * 60 + minute;

    bool peak =
        (t >= 480 && t < 600) ||
        (t >= 1020 && t < 1200);

    if (peak)
        amount += amount * 20 / 100;

    return amount;
}

int main()
{
    long long total = 0;
```

```
    int zones[] = {0, 1, 2, 0, 2};
    int hours[] = {12, 9, 18, 10, 20};
    int minutes[] = {0, 0, 30, 0, 0};

    for (int i = 0; i < 5; i++)
    {
        long long f = fare(zones[i], hours[i], minutes[i]);

        if (f == -1)
        {
            cout << "Invalid Journey" << endl;
        }
        else
        {
            cout << "Journey " << i + 1
                 << " Fare: " << f << " paise" << endl;

            total += f;
        }
    }

    cout << "Total Fare: " << total << " paise";

    return 0;
}
```

Expected output:

```
Journey 1 Fare: 1000 paise
Journey 2 Fare: 2400 paise
Journey 3 Fare: 3600 paise
Journey 4 Fare: 1000 paise
Journey 5 Fare: 3000 paise
Total Fare: 11000 paise
```

8. Alternative Concept: Table-Driven Testing

Instead of writing a separate test statement for every case, test cases can be stored in an array.
Each row contains the input and the expected fare.
The program loops through the array and compares the actual result with the expected result.
Example:

```
struct TestCase
{
    int zone;
    int hour;
    int minute;
    long long expected;
};
```

This method makes it easier to add new boundary cases without writing new testing logic.
Complexity Analysis:

- One fare calculation: O(1).
- One test execution: O(1).
- Testing t cases: O(t).
- Storage for t test cases: O(t).

Conclusion

Testing verifies that the metro fare program works correctly under different conditions. Normal, boundary and invalid test cases are necessary to identify errors. Table-driven testing is recommended because it makes the program easier to test and extend.

TASK 10 – COMPARE EXPECTATIONS WITH EXECUTION **4 Marks**

Question

Compare the expected and executed results for 20,000, 500,000 and 3,000,000 journeys. Discuss integer and floating-point totals, integer overflow, compiler warnings and the use of int, long long and double.

Answer

1. Introduction

A metro network may process a very large number of journeys every day. Therefore, the program must use appropriate data types to store individual fares and accumulated totals.

The main purpose of this task is to compare the behaviour of different data types when processing large numbers of journeys.

2. Journey Volume

The system is expected to support three levels of journey volume:

Stage	Number of journeys
Initial network	20,000 per day
Medium network	500,000 per day
Large network	3,000,000 per day

As the number of journeys increases, the total fare value also increases. A data type that works for a small number of journeys may not be suitable for a large network.

3. Comparison of Data Types

Integer (int)

The int data type commonly uses 4 bytes. Its typical maximum value is 2,147,483,647.

It may be sufficient for small calculations, but it can overflow when the total fare becomes very large.

Long long (long long)

The long long data type commonly uses 8 bytes. It supports a much larger range than int.

It is suitable for storing large accumulated fare values in paise.

Double (double)

The double data type is used to store decimal values.

It has greater precision than float, but decimal values are generally represented in binary floating-point format. Some decimal fractions cannot be represented exactly.

4. Illustrative Network Total Comparison

The following values are taken from the supplied reference solution. They are illustrative sample results and must not be described as independently measured results unless the experiment is reproduced.

Number of journeys	long long total in paise	double total converted to paise	Reference sample observation
20,000	37,563,600	37,563,600	Same total
500,000	937,370,000	937,370,000	Same total
3,000,000	5,626,462,900	5,626,462,900	Same total for the sample fares

The reference solution also reports a different result for a float accumulator at the largest journey volume because a float cannot retain every small increment once the accumulated value becomes large.

5. Integer Overflow

Integer overflow occurs when a value exceeds the range supported by the data type.

For a typical 32-bit signed int, the maximum positive value is:

\[2,147,483,647 \]

The reference solution's total for 3,000,000 journeys is:

\[5,626,462,900\text{ paise} \]

This value is greater than the maximum value of a typical 32-bit signed int.

Therefore, using int for the network total is unsafe.

In C++, signed integer overflow is undefined behaviour. A program must not rely on a particular wrapped result.

6. Floating-Point Precision

A float has less precision than a double.

When a large total is repeatedly updated with small fare values, some increments may not be represented exactly.

A double provides more precision, but it is still a floating-point type and should not be treated as an exact representation of every decimal monetary amount.

For financial calculations, integer paise is preferable.

7. Compiler Warnings

Compiler warnings help identify possible programming errors.

The reference solution uses:

g++ -std=c++17 -Wall -Wextra program.cpp -o program

Important warning categories include:

Warning	Meaning	Possible correction
-Wsign-compare	Signed and unsigned values are compared	Use compatible types
-Wsequence-point	Expression may modify a variable in an unsafe way	Avoid expressions such as i = i++

-Wunused-variable	A declared variable is not used	Remove or use the variable
-Wswitch	An enumeration value is missing from a switch	Add the missing case
Narrowing warning	Conversion may lose information	Use an appropriate type or explicit conversion

The actual warnings depend on the source code and compiler version. Record the warnings received when compiling your own programs.

8. Alternative Concept: Comparing int, long long and double

Data type	Advantages	Limitations
int	Uses less memory	Can overflow for large totals
long long	Large integer range and exact paise storage	Uses more memory than a typical int
double	Supports decimal calculations	Floating-point representation may introduce rounding
float	Uses relatively little memory	Lower precision

9. Recommended Implementation

```
#include <iostream>
using namespace std;

int main()
{
   long long totalPaise = 0;

   long long fare = 1200;

   int journeys = 3000000;

   for (int i = 0; i < journeys; i++)
   {
      totalPaise += fare;
   }

   cout << "Total fare in paise: "
        << totalPaise << endl;

   return 0;
}
```

This example demonstrates the use of long long for accumulating a large total.

Complexity Analysis:

- One addition: O(1).
- Processing n journeys: O(n).
- Storage for the total accumulator: O(1).
- Typical memory for long long: 8 bytes.

Conclusion

The data type used for fare accumulation must support the maximum expected network total. int may overflow for millions of journeys, while float may lose precision. long long integer paise is recommended because it provides a large range and exact storage of whole paise.

TASK 11 – RECOMMEND AND DEFEND THE FINAL DESIGN **4 Marks**

Question

Recommend the final design for the metro system. Discuss the data types, constants and classes, fare revision, limitations, scalability and evidence from experiments. Compare compile-time fare constants with a configuration file.

Answer

1. Introduction

The final metro smart-card system must be accurate, secure, maintainable and scalable. It must process journeys without incorrect deductions or negative balances.

The system should use suitable data types, named constants and object-oriented classes.

2. Recommended Data Types

Requirement	Recommended type	Reason
Fare amount	long long	Stores integer paise accurately

Network total	long long	Supports large accumulated totals
Zone	enum class Zone	Provides type-safe zone selection
Peak status	bool	Stores true or false
Hour and minute	int	Stores time values
Passenger concession	bool or suitable category type	Represents concession eligibility
Journey count	int or suitable unsigned size type	Stores the number of journeys

3. Recommended Constants

The fare values and rules should be maintained in one place.
Example:

```
constexpr long long INNER_FARE = 1000;
constexpr long long MIDDLE_FARE = 2000;
constexpr long long OUTER_FARE = 3000;

constexpr int PEAK_SURCHARGE_PERCENT = 20;

constexpr long long MIN_BALANCE_PAISE = 5000;
```

Using named constants avoids repeating values throughout the program.

4. Recommended Classes

SmartCard class
The SmartCard class stores the passenger's balance and provides functions to check and deduct fares.
Main functions:

- canTravel() – checks whether the journey is permitted.
- deduct() – deducts the fare safely.
- getBalance() – returns the current balance.
- recharge() – adds an approved recharge amount.

FareTable
The FareTable stores the base fare values and provides the correct fare for a selected zone.
Journey
The Journey structure stores journey information such as zone, time and concession status.

5. Recommended Peak-Hour Function

```
bool isPeak(int hour, int minute)
{
   int t = hour * 60 + minute;

   return (t >= 480 && t < 600) ||
          (t >= 1020 && t < 1200);
}
```

This function converts the time into minutes since midnight and checks both peak-hour intervals.

6. Fare Revision Procedure

When the metro authority changes a fare, the program should follow these steps:

1. Update the fare value in the central fare table.
2. Recompile the program using the required C++ standard.
3. Run all existing test cases.
4. Add new test cases for the revised fare.
5. Check the peak-hour and minimum-balance rules.
6. Deploy the tested program to the metro gates.

This avoids maintaining different fare values in different parts of the program.

7. Limitations of the Design

1. The original design supports only three zones.
2. It does not initially include weekend discounts.
3. It does not initially include monthly passes.
4. Fare changes require recompilation when values are compiled into the program.
5. A real metro system needs secure storage and transaction management.
6. Simultaneous transactions on the same card require synchronization and database protection.
7. Passenger concession rules must be confirmed by the metro authority.

8. Scalability

The use of long long supports large network totals.
Each journey requires a fixed number of operations, such as fare lookup, peak-hour checking and balance validation.

Therefore:

- One journey: O(1).
- n journeys: O(n).
- Fare table lookup: O(1).
- Smart-card balance storage: O(1) per card.

The system can support a large number of journeys if the overall software architecture, storage and transaction processing are also designed for the expected load.

9. Alternative Concept: Configuration File

Instead of storing fare values as compile-time constants, the system can read them from a configuration file.
Example file named fares.cfg:

```
1000
2000
3000
```

A C++ program can read these values using file handling.

```
#include <iostream>
#include <fstream>
#include <vector>
using namespace std;

int main()
{
   ifstream file("fares.cfg");

   vector<long long> fares;
   long long value;

   if (!file)
   {
      cout << "Unable to open fare file";
      return 1;
   }

   while (file >> value)
   {
      if (value <= 0)
      {
         cout << "Invalid fare value";
         return 1;
      }

      fares.push_back(value);
   }

   if (fares.size() != 3)
   {
      cout << "Invalid fare configuration";
      return 1;
   }

   cout << "Fare values loaded successfully";

   return 0;
}
```

Output:

```
Fare values loaded successfully
```

10. Compile-Time Constants versus Configuration File

Feature	Compile-time constants	Configuration file
Fare storage	In program source	External file
Fare update	Requires source update and rebuild	File can be updated

Startup operation	No file loading	Reads and validates file
Runtime errors	Fewer configuration-file errors	File may be missing or invalid
Maintenance	Requires deployment after changes	Easier for frequent changes
Flexibility	Lower	Higher

Complexity Analysis:

If there are z fare values:

- Loading the configuration file: O(z).
- Storing the values: O(z).
- Looking up one fare: O(1).
- Processing n journeys after loading: O(n).

Conclusion

The recommended design uses long long integer paise, constexpr constants, enum class Zone, a FareTable and a SmartCard class. This design provides accurate fare calculation, safe balance management and easier maintenance. A configuration file can be introduced when the metro authority requires frequent fare revisions without rebuilding the application.

TASK 12 – RE-EVALUATE AND ADAPT THE DESIGN

8 Marks

Changed Requirements

The metro network is extended with three new requirements:

1. A fourth zone named Airport with a base fare of ₹60.
2. A 10% weekend discount that does not combine with the peak-hour surcharge.
3. A monthly pass that allows unlimited journeys in two selected zones.

The original design must be evaluated and modified to support these changes.

Answer

A. Impact Analysis

1. Introduction

Impact analysis identifies which parts of the existing system must be modified when new requirements are introduced.

The original system supports three zones, peak-hour surcharges and smart-card balance deductions. The new requirements introduce an additional zone, a weekend discount and a monthly pass.

These changes affect the fare table, zone enumeration, fare calculation function, journey information and smart-card design.

2. Changes to Constants

The original base fare values are:

```
constexpr long long BASE_FARE[] = {
    1000, 2000, 3000
};
```

The new Airport zone requires an additional value:

```
constexpr long long BASE_FARE[] = {
    1000, 2000, 3000, 6000
};
```

A new weekend discount constant is also required:

```
constexpr int WEEKEND_DISCOUNT_PERCENT = 10;
```

3. Changes to Enumeration

Original enumeration:

```
enum class Zone
{
    Inner,
    Middle,
    Outer
};
```

Modified enumeration:

```
enum class Zone
{
    Inner,
    Middle,
    Outer,
    Airport,
    Count
```

};
The new Airport value represents the fourth zone. Count helps determine the number of zones when sizing the fare table.

4. Changes to Fare Calculation

The original fare calculation applies a peak surcharge when the journey occurs during peak hours.
The revised system must apply either:

- A 10% weekend discount, or
- A 20% weekday peak surcharge.

The two adjustments must not be combined.

5. Changes to SmartCard

The original SmartCard class stores only the balance.
The revised system must also support monthly pass information. A journey covered by a valid pass should be free and should not deduct money from the balance.

6. Impact Analysis Table

Original component	Change required	Reason
Base fare table	Add ₹60 Airport fare	Fourth zone
Zone enumeration	Add Airport	New travel zone
Peak surcharge	Retain and modify its application	Must not combine with weekend discount
Weekend rule	Add 10% discount	New requirement
Journey structure	Add weekday and month information	Required for discount and pass validation
SmartCard class	Add pass checking	Pass-covered journeys are free
Pass class	Introduce new class	Stores two selected zones and validity month
Input validation	Extend zone range	Must accept Airport

Conclusion for Impact Analysis
The new requirements affect the fare calculation, journey data and smart-card design. A separate Pass class and an updated fare table are required to support the new features correctly.

B. Evaluation of the Original Design

1. Evaluation of the Switch-Based Fare Logic

The original system uses a switch statement to select one of three base fares.
When the Airport zone is added, a new case must be added.
If the Airport case is forgotten, the fare function may fail to calculate the correct fare. The compiler can warn that an enumeration value is not handled, but the programmer must still correct the code.
A fare table indexed by the zone is easier to extend because a new fare value can be added to the table.

2. Evaluation of Weekend Discount

The original fare function receives only the zone and peak-hour status.
However, the revised system also needs the day of the week.
Therefore, the function must be modified to receive additional journey information.
The weekend discount must replace the peak surcharge according to the changed requirement.

3. Worked Example: Airport Weekday

Airport base fare:
$$₹60$$
Peak surcharge:
$$₹60 \times \frac{20}{100}=₹12$$
Final fare:
$$₹60+₹12=₹72$$

4. Worked Example: Airport Weekend

Airport base fare:
$$₹60$$
Weekend discount:
$$₹60 \times \frac{10}{100}=₹6$$
Final fare:
$$₹60-₹6=₹54$$
The peak surcharge is not applied on the weekend.

5. Worked Example: Student Airport Weekend

The supplied reference solution assumes a 50% passenger concession. This percentage is an assumption because the original assignment does not specify it.
Airport base fare:
$$₹60$$
After 50% concession:

\[₹60-₹30=₹30 \]
After 10% weekend discount:
\[₹30-₹3=₹27 \]
Final fare:
\[₹27 \]
The order of applying concessions and discounts must be confirmed by the metro authority.

6. Evaluation of Monthly Pass

A monthly pass allows unlimited journeys in two selected zones.
For example, a passenger selects:

- Inner Zone.
- Middle Zone.

A journey in either covered zone during the valid month should be free.
A journey in the Outer or Airport zone is not covered by the pass and must be charged normally.

7. Original Design Evaluation Table

Requirement	Original design	Required modification
Three zones	Supported	No change to existing zones
Airport zone	Not supported	Add fourth zone
Weekend discount	Not supported	Add weekend rule
Monthly pass	Not supported	Add Pass class
Smart-card balance	Supported	Add pass validation
Peak surcharge	Supported	Ensure it does not apply on weekends

Conclusion for Evaluation
The original design provides a suitable foundation, but it cannot support the new requirements without modification. The fare function, zone table and SmartCard class must be extended.

C. Adaptation and Redesigned System

1. Proposed Design

The revised system uses:

1. An array-based fare table for four zones.
2. A Journey structure to store zone, time, weekday and month.
3. A separate fare function to calculate the correct fare.
4. A Pass class to store two selected zones and monthly validity.
5. A SmartCard class to maintain balance and apply the pass rules.

2. Revised Fare Rules

- Inner Zone: ₹10.
- Middle Zone: ₹20.
- Outer Zone: ₹30.
- Airport Zone: ₹60.
- Weekday peak surcharge: 20%.
- Weekend discount: 10%.
- Weekend discount replaces the peak surcharge.
- The reference solution assumes a 50% concession for eligible passengers.
- A valid pass makes journeys in the selected zones free for the valid month.

3. Revised C++ Program

The following is a model implementation of the revised design. It uses integer paise and keeps the balance private.

```
#include <iostream>
#include <string>
using namespace std;

enum class Zone
{
   Inner,
   Middle,
   Outer,
   Airport,
   Count
};

constexpr int ZONE_COUNT =
```

```
    static_cast<int>(Zone::Count);

constexpr long long BASE_PAISE[ZONE_COUNT] =
{
    1000, 2000, 3000, 6000
};

constexpr int PEAK_PERCENT = 20;
constexpr int WEEKEND_PERCENT = 10;
constexpr int CONCESSION_PERCENT = 50;

constexpr long long MIN_BALANCE = 5000;

struct Journey
{
    Zone zone;
    int hour;
    int minute;
    int weekday;  // 0 = Monday, 6 = Sunday
    bool concession;
    int year;
    int month;
};

bool isWeekend(int weekday)
{
    return weekday == 5 || weekday == 6;
}

bool isPeak(int hour, int minute)
{
    int t = hour * 60 + minute;

    return (t >= 480 && t < 600) ||
           (t >= 1020 && t < 1200);
}

long long percentage(long long amount, int percent)
{
    return (amount * percent + 50) / 100;
}

long long calculateFare(const Journey& j)
{
    int zoneIndex = static_cast<int>(j.zone);

    if (zoneIndex < 0 || zoneIndex >= ZONE_COUNT)
        return -1;

    if (j.hour < 0 || j.hour > 23 ||
        j.minute < 0 || j.minute > 59 ||
        j.weekday < 0 || j.weekday > 6)
        return -1;

    long long fare = BASE_PAISE[zoneIndex];

    // Assumption: concession is applied first.
    if (j.concession)
    {
        fare -= percentage(fare, CONCESSION_PERCENT);
```

```
    }

    // Weekend discount replaces peak surcharge.
    if (isWeekend(j.weekday))
    {
        fare -= percentage(fare, WEEKEND_PERCENT);
    }
    else if (isPeak(j.hour, j.minute))
    {
        fare += percentage(fare, PEAK_PERCENT);
    }

    return fare;
}

class Pass
{
private:
    unsigned int zoneMask;
    int validYear;
    int validMonth;

public:
    Pass(Zone first, Zone second, int year, int month)
    {
        zoneMask =
            (1u << static_cast<int>(first)) |
            (1u << static_cast<int>(second));

        validYear = year;
        validMonth = month;
    }

    bool covers(const Journey& j) const
    {
        if (j.year != validYear ||
            j.month != validMonth)
            return false;

        int index = static_cast<int>(j.zone);

        if (index < 0 || index >= ZONE_COUNT)
            return false;

        return (zoneMask & (1u << index)) != 0;
    }
};

class SmartCard
{
private:
    long long balance;
    const Pass* monthlyPass;

public:
    SmartCard(long long initialBalance)
        : balance(initialBalance), monthlyPass(nullptr)
    {
    }
```

```
    void attachPass(const Pass* p)
    {
        monthlyPass = p;
    }

    long long getBalance() const
    {
        return balance;
    }

    long long travel(const Journey& j)
    {
        // A valid pass covers this journey.
        if (monthlyPass != nullptr &&
            monthlyPass->covers(j))
        {
            return 0;
        }

        long long fare = calculateFare(j);

        if (fare < 0)
            return -1;

        if (balance - fare < MIN_BALANCE)
            return -1;

        balance -= fare;

        return fare;
    }
};

string formatRupees(long long paise)
{
    string result = to_string(paise / 100);

    result += ".";

    if (paise % 100 < 10)
        result += "0";

    result += to_string(paise % 100);

    return result;
}

int main()
{
    Pass monthlyPass(
        Zone::Inner,
        Zone::Middle,
        2026,
        11
    );

    SmartCard card(10000);

    card.attachPass(&monthlyPass);
```

```
    Journey j1 =
    {
        Zone::Airport, 12, 0, 0,
        false, 2026, 11
    };

    Journey j2 =
    {
        Zone::Airport, 9, 0, 5,
        true, 2026, 11
    };

    Journey j3 =
    {
        Zone::Middle, 9, 0, 0,
        false, 2026, 11
    };

    long long f1 = card.travel(j1);
    long long f2 = card.travel(j2);
    long long f3 = card.travel(j3);

    if (f1 >= 0)
        cout << "Airport weekday fare: Rs "
             << formatRupees(f1) << endl;
    else
        cout << "Journey 1 denied" << endl;

    if (f2 >= 0)
        cout << "Airport weekend student fare: Rs "
             << formatRupees(f2) << endl;
    else
        cout << "Journey 2 denied" << endl;

    if (f3 == 0)
        cout << "Middle journey covered by monthly pass"
             << endl;
    else if (f3 > 0)
        cout << "Middle journey fare: Rs "
             << formatRupees(f3) << endl;
    else
        cout << "Journey 3 denied" << endl;

    cout << "Remaining balance: Rs "
         << formatRupees(card.getBalance())
         << endl;

    return 0;
}
```

4. Expected Output

The first journey is an Airport weekday off-peak journey.
Airport base fare:
$$ ₹60 $$
The second journey is an Airport weekend journey with the assumed 50% concession.
$$ ₹60 \times 50\% = ₹30 $$
Weekend discount:
$$ ₹30 \times 10\% = ₹3 $$
Final fare:
$$ ₹30-₹3=₹27 $$
The third journey is in the Middle Zone, which is covered by the valid monthly pass.

Expected output:
Airport weekday fare: Rs 60.00
Airport weekend student fare: Rs 27.00
Middle journey covered by monthly pass
Remaining balance: Rs 13.00
Note: The above output assumes the card starts with ₹100 and the ₹50 minimum balance rule applies to paid journeys. After the first two paid journeys, the balance is ₹13, so this example would actually violate the minimum-balance rule for the second paid journey. The code correctly rejects that journey. To demonstrate both paid journeys successfully, change the starting balance to ₹150.
With SmartCard card(15000);, the expected output is:
Airport weekday fare: Rs 60.00
Airport weekend student fare: Rs 27.00
Middle journey covered by monthly pass
Remaining balance: Rs 63.00

5. Test Cases for the Revised System

Test case	Input	Expected result
Airport weekday off-peak	Airport, Monday, 12:00	₹60
Airport weekday peak	Airport, Monday, 09:00	₹72
Inner weekend off-peak	Inner, Saturday, 12:00	₹9
Inner weekend during peak window	Inner, Saturday, 09:00	₹9
Outer Sunday	Outer, Sunday, 18:00	₹27
Airport Sunday with assumed concession	Airport, Sunday, 12:00	₹27
Valid monthly pass	Middle, valid month	Free
Pass not covering zone	Outer, valid month	Normal fare
Expired monthly pass	Middle, different month	Normal fare
Invalid zone	Zone outside 0–3	Rejected

6. Important Design Considerations

1. The fare table must contain exactly four zone values.
2. The weekend discount must not combine with the peak surcharge.
3. The monthly pass must check both the selected zone and validity month.
4. The balance must remain private inside the SmartCard class.
5. Invalid dates, months, zones and times must be rejected.
6. The concession percentage and calculation order must be confirmed with the metro authority.
7. The pass should be stored safely in a real system rather than relying on an unmanaged pointer.
8. The fare calculation should remain in one function to avoid duplicated logic.

D. Evidence-Based Defence

1. Why the Revised Design Is Suitable

The revised design retains the useful features of the original system while adding support for the new requirements.
The following features are retained:

- Integer paise for accurate money calculations.
- Named constants for fare values and percentages.
- Private balance in the SmartCard class.
- A separate peak-hour checking function.
- A minimum balance validation rule.

The following features are added:

- Fourth Airport zone.
- Weekend discount.
- Journey date and weekday information.
- Monthly Pass class.
- Pass coverage validation.

2. Advantages of the Revised Design

1. Supports four travel zones.
2. Avoids duplicated fare values.
3. Supports weekend discounts.
4. Prevents the weekend discount and peak surcharge from being combined.
5. Supports unlimited travel in two selected zones during the valid month.
6. Maintains the minimum-balance rule for paid journeys.
7. Uses encapsulation to protect card balance.

8. Makes future changes easier.

3. Trade-Offs

Design choice	Advantage	Limitation
Fare array	Easy to add a new zone	Must validate the index
Separate fare function	Centralizes fare rules	Must be updated for new fare policies
Pass class	Keeps pass logic separate	Needs correct validity information
Private balance	Protects card balance	Requires member functions
Integer paise	Accurate whole-paise calculations	Conversion is needed for display

4. Complexity Analysis

- Fare-table lookup: O(1).
- Peak-hour checking: O(1).
- Weekend checking: O(1).
- Pass coverage checking: O(1) for two selected zones using a bit mask.
- Smart-card balance update: O(1).
- Processing n journeys: O(n).

5. Final Recommendation

The revised design is recommended because it supports the fourth Airport zone, weekend discounts and monthly passes without changing the entire program structure.

The system maintains the important features of the original design, such as integer paise, encapsulation and minimum-balance validation. The use of separate classes and functions makes the software easier to maintain and extend.

Conclusion

The original metro fare system was designed for three zones and peak-hour surcharges. The revised design extends the system to four zones, adds a 10% weekend discount and introduces monthly passes for two selected zones. The new design uses an array-based fare table, a Journey structure, a Pass class and a SmartCard class. This provides a more flexible and scalable solution for the expanded metro network.

OVERALL CONCLUSION

The City Metro Smart-Card System demonstrates how C++ Object-Oriented Programming concepts can be applied to a real-world fare management problem.

The assignment covers the use of classes, objects, encapsulation, constants, data types, operators, control structures, input validation and testing. The system calculates fares based on travel zones and peak-hour conditions while ensuring that the remaining smart-card balance does not fall below ₹50.

The use of long long for integer paise avoids many financial rounding problems and supports large fare totals. The SmartCard class protects the balance through private data members and controlled member functions. Testing normal, boundary and invalid inputs helps identify errors and improve reliability.

The extended design supports the Airport zone, weekend discounts and monthly passes. These improvements make the system more flexible and easier to maintain. Overall, the proposed system provides a foundation for a reliable and scalable metro smart-card fare management application.

REFERENCES

1. Bjarne Stroustrup, The C++ Programming Language, 4th Edition, Addison-Wesley.
2. Bjarne Stroustrup, A Tour of C++, 3rd Edition, Addison-Wesley.
3. ISO/IEC 14882, Programming Languages – C++ Standard.
4. cppreference.com, “C++ Language Reference.” https://en.cppreference.com/
5. GCC Documentation, “Warning Options.” https://gcc.gnu.org/onlinedocs/gcc/Warning-Options.html
6. ISO C++ Foundation, “Standard C++.” https://isocpp.org/
7. Microsoft Learn, “C++ Language Documentation.” https://learn.microsoft.com/en-us/cpp/
8. C++ Reference, “Input/Output Library.” https://en.cppreference.com/w/cpp/io
9. C++ Reference, “Containers Library.” https://en.cppreference.com/w/cpp/container
10. C++ Reference, “Classes and Objects.” https://en.cppreference.com/w/cpp/language/classes